Введение в dbt (data build tool)

Цели курса

- Понять ключевые концепции **dbt**
- Изучить практические применения
- Научиться создавать и настраивать **dbt-проект**
- Писать и запускать базовые **dbt-модели**
- Освоить один из самых востребованных инструментов в data engineering

Часть 1

Знакомство с dbt

Что такое dbt?

dbt = "data build tool"

- Отвечает за **T** (Transform) в пайплайне **ELT**
- Позволяет легко переключаться между хранилищами данных
  - Snowflake, BigQuery, PostgreSQL, DuckDB, ClickHouse...
- Обеспечивает совместную SQL-разработку в команде
- Транслирует SQL между диалектами баз данных

```
ETL:  Источник  →  Трансформация  →  Загрузка в хранилище
ELT:  Источник  →  Загрузка "как есть"  →  Трансформация (dbt!)
```

Что делает dbt?

- Определяет **модели данных** и трансформации на SQL
  - E-commerce, мобильные приложения, IoT…
- Задаёт **связи** между моделями
  - Клиенты, Адреса, Заказы
- **Тестирует** качество данных (Data Quality)

Как выглядит dbt?

dbt Core (open-source)

- CLI-инструмент
- Доступен для Mac, Windows, Linux
- Устанавливается через `pip`

dbt Cloud (проприетарный)

- Веб-интерфейс
- Управляемая среда
- IDE в браузере

```
# Ключевые команды
dbt --version    # проверка версии
dbt --help       # справка
```

Для кого dbt?

dbt предназначен для всех, кому нужно **трансформировать данные**:

Роль	Как использует dbt
Data Engineer	Построение пайплайнов, оркестрация
Analytics Engineer	Моделирование, метрики, документация
Data Analyst	Ad-hoc трансформации, отчёты

> ***Analytics Engineer*** *-- роль, которая возникла благодаря dbt. Это мост между data engineering и аналитикой: создаёт модели данных, описывает метрики, пишет тесты качества, ведёт документацию и обеспечивает надёжный self-service доступ к данным для аналитиков и бизнеса.*

Часть 2

Создание dbt-проекта

Что такое dbt-проект?

Набор файлов, определяющих **как** работать с данными:

- Конфигурация проекта
- Источники и назначения данных
- SQL-запросы и шаблоны
- Документация
- Тесты

```
NYC_YELLOW_TAXI/
  analyses/
  dbt_packages/
  logs/
  macros/
  models/
    taxi_rides/
      model_properties.yml
  seeds/
  snapshots/
  target/
  tests/
```

Создание нового проекта

Используем команду `dbt init`:

```
repl:~/workspace$ dbt init
16:38:37  Running with dbt=1.8.4
Enter a name for your project (letters, digits, underscores): test_project
Which database would you like to use? [1] duckdb
Enter a number: 1
```

- Спрашивает **имя проекта** и **тип базы данных**
- Можно сконсолидировать: `dbt init <projectname>`
- Создаёт всю структуру папок и файлов

Профили проекта (`profiles.yml`)

Профиль = конфигурация среды выполнения

- **Development** -- локальная разработка
- **Staging / Test** -- тестовая среда
- **Production** -- продакшн

Один проект может иметь **несколько профилей**

```yaml
marketing_campaign_results:
  outputs:
    dev:
      type: duckdb
      path: dbt.duckdb
    prod:
      type: snowflake
      ...

  target: dev
```

DuckDB -- быстрый старт

- Open-source, **встроенная** (serverless) СУБД
- Аналог SQLite, но для **аналитики**
- **Векторизованный** движок -- очень быстрый
- Не требует сервера
- Идеален для обучения и прототипирования

DuckDB

```
-- DuckDB умеет читать файлы напрямую
SELECT * FROM 'data.parquet';
SELECT * FROM 'data.csv';
SELECT * FROM read_json('api.json');
```

Часть 3

Работа с первым проектом

Рабочий процесс dbt

```
1. Создать проект           dbt init
2. Настроить конфигурацию  profiles.yml
3. Создать модели           SQL-файлы в models/
4. Материализовать          dbt run
5. Проверить / Отладить     dbt test
6. Повторить
```

Большую часть времени вы будете проводить на **шагах 3-6** -- итеративная разработка моделей.

`dbt run`

Главная команда -- запускает трансформации:

```
repl:~$ dbt run
04:52:11  Running with dbt=1.8.4
04:52:13  1 of 1 START sql view model main.sales_data ...... [RUN]
04:52:13  1 of 1 OK created sql view model main.sales_data . [OK in 0.12s]
04:52:13  Completed successfully
```

- Выполняется при **изменении моделей** или необходимости материализации
- Показывает детали **успеха/ошибок** каждого шага
- **Материализация** = превращение SQL в таблицы/представления

Материализация в dbt

По умолчанию dbt создаёт **представления (views)**, а не таблицы.

Чтобы создать **таблицу**, укажите материализацию в конфиге модели:

```
-- models/taxi/avg_fare_per_day.sql
{{ config(materialized='table') }}

SELECT
    pickup_date,
    AVG(fare_amount) AS avg_fare
FROM taxi_rides_raw
GROUP BY pickup_date
```

Или глобально: в `dbt_project.yml` добавьте `+materialized: table` в секцию `models`

Часть 4

Модели данных в dbt

Что такое модель в dbt?

- Представляет **трансформации** над исходными данными
- SQL-модель -- это **всегда SELECT-запрос**
- Также возможны **Python-модели** (с dbt 1.3+)
- Каждая модель = отдельный `.sql` или `.py` файл

```sql
-- models/taxi/avg_fare_per_day.sql
SELECT
    pickup_date,
    AVG(fare_amount) AS avg_fare
FROM taxi_rides_raw
GROUP BY pickup_date
```

Создание простой модели

4 шага:

1. Создать директорию в `models/`
2. Создать `.sql` файл
3. Написать SELECT-запрос
4. Выполнить `dbt run`

```
mkdir models/order
touch models/order/customer_orders.sql
```

```
-- models/order/customer_orders.sql
SELECT
    first_name,
    last_name,
    shipping_address,
    item_quantity
FROM source_table
```

```
dbt run
```

Чтение из Parquet

Parquet -- колоночный бинарный формат для эффективного хранения

- Широко используется в data engineering
- DuckDB читает Parquet **напрямую**
- Два способа обращения:

```sql
-- Через функцию
SELECT *
FROM read_parquet('filename.parquet')

-- Через имя файла в кавычках
SELECT *
FROM 'filename.parquet'
```

> *Parquet vs CSV: в 2-10x компактнее и значительно быстрее благодаря колоночному хранению и сжатию*

Часть 5

Обновление моделей

Обновление моделей

Шаг	Действие	Команда
1	Клонировать проект	`git clone`
2	Обновить модель	--
3	Перегенерировать	`dbt run`
4	Закоммитить	`git commit`

Флаг `--full-refresh` (`-f`) — пересоздаёт модель с нуля (дропает и строит заново). Нужен при изменении **схемы** или **логики**, затрагивающей исторические данные.

Конфигурационные YAML-файлы

`dbt_project.yml`

- Настройки **всего проекта**
- Имя, версия, пути
- Глобальная материализация
- **Один файл** на проект

```yaml
name: 'my_project'
version: '1.0.0'
models:
  my_project:
    +materialized: view
```

`model_properties.yml`

- Настройки **конкретных моделей**
- Описания и документация
- Лежит в `models/`
- **Сколько угодно** файлов

```yaml
version: 2
models:
  - name: taxi_rides_raw
    description: Yellow Taxi raw data
    columns:
      - name: avg_fare_per_day
        description: Average fare
```

Часть 6

Документация

Документация в dbt

dbt **автоматически** добавляет документацию в проект:

- Описания моделей и колонок
- Lineage / DAG (граф зависимостей)
- Тесты и валидации
- Информация из хранилища
  - Типы данных
  - Размеры таблиц

```yaml
version: 2
models:
  - name: taxi_rides_raw
    description: Yellow Taxi raw data
    access: public
    columns:
      - name: avg_fare_per_day
        description: Average ride fare
        access: public
```

Пример документации dbt

Веб-интерфейс `dbt docs serve` показывает:

Раздел	Что содержит
View / Models	Список всех моделей проекта
Description	Описание из `model_properties.yml`
Column details	Типы данных, описания колонок
Lineage graphs	Визуальный граф зависимостей (DAG)

> *Lineage-граф -- одна из самых полезных возможностей. Позволяет видеть, какие модели зависят друг от друга.*

Генерация и просмотр документации

```
dbt docs generate     # создать документацию
dbt docs serve        # запустить локальный веб-сервер
```

Варианты хостинга:

- `dbt docs serve` -- только для локальной разработки
- **dbt Cloud** -- встроенный хостинг
- **Amazon S3 / Nginx / Apache** -- для продакшна

Часть 7

Шаблоны Jinja

Что такое шаблон?

- Формат с **подстановкой** переменных
- Обычно -- текстовый файл
- Упрощает **переиспользование** кода

Что такое Jinja?

- Текстовый шаблонизатор (Python-экосистема)
- Используется в **dbt**, Django, Flask, Airflow
- `{{ ... }}` -- подстановка значений
- `{% ... %}` -- логика (циклы, условия)

Функции Jinja в dbt

Функция	Назначение
`ref()`	Ссылка на другую модель по имени
`config()`	Доступ к настройкам
`docs`	Доступ к документации
`source()`	Ссылка на источник данных

Также доступны: математические операции, циклы, условия, макросы

Пример Jinja

Без Jinja

```
SELECT
  COALESCE(start_date,
    '2025-01-01') AS start_date,
  COALESCE(update_date,
    '2025-01-01') AS update_date,
  COALESCE(end_date,
    '2025-01-01') AS end_date
FROM Events
```

Повторяющийся код для каждой колонки

С Jinja

```
SELECT
  {% for column in [
    'start_date',
    'update_date',
    'end_date'] %}
  COALESCE({{ column }},
    '2025-01-01')
    AS {{ column }}
    {% if not loop.last %},{% endif %}
  {% endfor %}
FROM Events
```

DRY -- цикл устраняет дублирование

Часть 8

Иерархические модели

Что такое иерархия в dbt?

- Описывает **зависимости** между моделями
- Также известна как **DAG** (Directed Acyclic Graph)
- Определяет **порядок сборки** моделей

```
taxi_rides_raw  -->  avg_fare_per_day
                -->  total_creditcard_rides_per_day
```

Без иерархии dbt строит модели в алфавитном порядке -- это приведёт к ошибке, если зависимость ещё не создана.

Определение зависимостей

Используем функцию `ref()` в Jinja:

Прямая ссылка (хрупко)

```
SELECT
    first_name, last_name
FROM taxi_rides_raw
```

Через `ref()` (правильно)

```
SELECT
    first_name, last_name
FROM {{ ref('taxi_rides_raw') }}
```

- `ref()` заменяется на реальное имя таблицы при `dbt run`
- dbt автоматически строит **граф зависимостей**
- Гарантирует правильный **порядок материализации**

> *`ref()` -- самая важная функция в dbt. Она не только заменяет имя, но и сообщает о зависимости.*

Итоги

Что мы изучили

Тема	Ключевые моменты
Что такое dbt	ELT-инструмент, SQL-трансформации
Проект	`dbt init`, `profiles.yml`, структура папок
Модели	`.sql` файлы, `dbt run`, таблицы vs views
Обновление	Итеративный цикл, `dbt run -f`
Документация	`dbt docs generate`, lineage graph
Jinja	`{{ }}`, `{% %}`, циклы, `ref()`
Иерархия	DAG, `ref()`, порядок сборки